

Vulnerability Research - Assignment

1. Write a NASM program that adds two numbers and displays the result.

The NASM Calculator code can be found here:

https://github.com/CharEmsW/Vulnerability_Research.git

```
1 section .data
2     welcome db 0dh, 0ah, 0dh, 0ah, "***Hey, welcome to this calculator!***", 0dh, 0ah
3     welcome_length equ $ - welcome
4     msg_prompt db "Add your calculation (e.g., 24 + 6): ", 0
5     msg_result db "Result: ", 0
6     msg_newline db 10, 0
7     invalid_operator db "Invalid input, please use + or - for operations", 0
8
9
10 section .bss
11     buffer resb 32
12     num1 resq 1 ; Number 1
13     num2 resq 1 ; Number 2
14
15 section .text
16     global _start
17
18     _start:
19     ; Print the welcome message
20     mov rsi, welcome
21     mov rdx, welcome_length
22     mov rax, 1
23     mov rdi, 1
24     syscall
25
26
27     ; Print the message prompt
28     mov rsi, msg_prompt
29     mov rdx, 37
30     mov rax, 1
31     mov rdi, 1
32     syscall
33
34     ; Read the input
35     mov rsi, buffer
36     mov rdx, 32
37     mov rax, 0
38     mov rdi, 0
39     syscall
40
41     ; Convert first number to an int
42     mov rsi, buffer
43     call parse_number
44     mov [num1], rax
45
46
```

```
47     ; Move to the operator (adjusting rsi based on parsed length)
48     add rsi, rax
49     call skip_spaces
50
51     ; Check the operator
52     movzx rbx, byte [rsi]
53     cmp rbx, '+'
54     je add_numbers
55     cmp rbx, '-'
56     je subtract_numbers
57
58     ; Invalid operator - print error message
59     mov rsi, invalid_operator
60     mov rdx, 47
61     mov rax, 1
62     mov rdi, 1
63     syscall
64     jmp _exit
65
66     ; Move to the 2nd
67     add_numbers:
68     inc rsi
69     call skip_spaces
70     call parse_number
71     mov [num2], rax
72
73     ; Perform addition
74     mov rax, [num1]
75     add rax, [num2]
76     call print_result
77     jmp _exit
78
79     ; Move to the 2nd
80     subtract_numbers:
81     inc rsi
82     call skip_spaces
83     call parse_number
84     mov [num2], rax
85
86     ; Perform subtraction
87     mov rax, [num1]
88     sub rax, [num2]
89     call print_result
90     jmp _exit
91
92 ; Example to add numbers section
```

```

Welcome calc.asm
home > cfg-vr > Desktop > Assignments > NasmCalc > calc.asm
92 ; Example in add numbers section
93 mov rax, [num1] ; Load num1
94 add rax, [num2] ; Perform addition, result in rax
95 call print_result ; Call print_result to display the result
96 jmp _exit
97
98 print_result:
99 ; Printing the results
100 mov rsi, msg_result
101 mov rdx, 8
102 mov rax, 1
103 mov rdi, 1
104 syscall
105
106 ; Converting results to a string
107 call print_number
108 ret
109
110 parse_number:
111 ; Converting a string to a number
112 xor rax, rax
113 mov rcx, 0
114
115 parse_loop:
116 movzx rbx, byte [rsi]
117 test rbx, rbx
118 jz done_parsing
119 cmp rbx, '0'
120 jb done_parsing
121 cmp rbx, '9'
122 ja done_parsing
123 sub rbx, '0'
124 imul rax, rax, 10
125 add rax, rbx
126 inc rsi
127 inc rcx
128 jmp parse_loop
129 done_parsing:
130 mov rax, rcx
131 ret
132
133 print_number:
134 ; Convert the number in rax to ASCII in buffer and display
135 mov rsi, buffer + 32
136 mov rcx, 10
137 add rsi, 1

```

```

Welcome calc.asm
home > cfg-vr > Desktop > Assignments > NasmCalc > calc.asm
138 print_number:
139 mov byte [rsi], 0
140
141 ; If the number is 0, handle it separately
142 cmp rax, 0
143 je print_zero
144
145 print_loop:
146 xor rdx, rdx
147 div rcx
148 dec rsi
149 add dl, '0'
150 mov [rsi], dl
151 test rax, rax
152 jnz print_loop
153 jmp print_result_buffer
154
155 print_zero:
156 dec rsi
157 mov byte [rsi], '0'
158 jmp print_result_buffer
159
160 print_result_buffer:
161 ; display result from buffer
162 mov rdx, buffer + 32
163 sub rdx, rsi
164 mov rax, 1
165 mov rdi, 1
166 syscall
167 ret
168
169 skip_spaces:
170 ; skipping any spaces in the buffer
171 skip_space_loop:
172 movzx rbx, byte [rsi]
173 cmp rbx, ' '
174 jne skip_done
175 inc rsi
176 jmp skip_space_loop
177 skip_done:
178 ret
179
180 _exit:
181 ; Exit the program
182 mov rax, 60
183 xor rdi, rdi
184 syscall

```

To compile the code I ran the following:

```

cfg-vr@codefirstgirls:~/Desktop/Assignments/NasmCalc$ nasm -f elf64 -o calc calc.asm
cfg-vr@codefirstgirls:~/Desktop/Assignments/NasmCalc$ ld -m elf_x86_64 -o calc calc.o
cfg-vr@codefirstgirls:~/Desktop/Assignments/NasmCalc$ ./calc

```

Then to run the program you input ./calc:

```

cfg-vr@codefirstgirls:~/Desktop/Assignments/NasmCalc$ ./calc

```

The following will display on the console:

```

**Hey, welcome to this calculator!**
Add your calculation (e.g., 24 + 6):

```

You type the calculation (+ or -) you'd like to run and the result **should** display:

```

**Hey, welcome to this calculator!**
Add your calculation (e.g., 24 + 6): 2 + 5
Result: cfg-vr@codefirstgirls:~/Desktop/Assignments/NasmCalc$

```

If an incorrect operator was input, the following would display:

```
**Hey, welcome to this calculator!**  
Add your calculation (e.g., 24 + 6): 3 / 4  
Invalid input, please use + or - for operationscfg-vr@codefirstgirls:~/Desktop/Assignments/NasmCalc$  
0 A 0 W 0
```

Numbers that are parsed from user input are stored in the variables num1 and num2 here:

```
section .bss  
    buffer resb 32  
    num1 resq 1 ; Number 1  
    num2 resq 1 ; Number 2
```

b. Perform the addition using add instruction;

```
add_numbers:  
    inc rsi  
    call skip_spaces  
    call parse_number  
    mov [num2], rax  
  
    ; Perform addition  
    mov rax, [num1]  
    add rax, [num2]  
    call print_result  
    jmp _exit
```

2. Based on the provided vulnerability database resource, pick two common vulnerabilities of your choice and explain:

CWE-244: Improper clearing of heap memory before release / Heap Inspection

Using `realloc()` to resize buffers that store sensitive information can leave that information exposed to attack as it has not been removed from memory.

When sensitive data, such as a password or an encryption key is not removed from memory, it could be then exposed to an attacker using a heap inspection attack that could read the data using various methods, such as (but not limited to):

- **DMA Attack (Direct memory attack)** - Using certain hardware devices to access system memory bypassing the CPU
- **Cold Boot Attack** - Forced reboot of the system and dumping the memory contents. Attacker can reboot into a custom OS to retrieve sensitive data remaining in memory
- **Rowhammer Attack** - A technique that manipulates DRAM hardware to cause bit flips in adjacent rows of memory cells, allowing an attacker to alter data or access restricted data by flipping specific bits. This manipulation can, for instance, flip bits in memory locations containing sensitive data or weaken memory protections.
- **Process Injection** - Injecting code into a legitimate process and using that to access the memory of that process. Techniques are used to allow attackers to read sensitive data in the memory of the applications they infiltrate.

`realloc()` is commonly used to increase the size block of allocated memory. This requires copying the contents of the old memory block into a newer and larger block, leaving the contents from the old one intact and inaccessible to the program (meaning you can't clear the data from memory) If an attacker can later examine the contents of the memory dump, data will be exposed.

```
1 char *cleartext_buffer = get_secret(); // This function retrieves sensitive data
2 cleartext_buffer = realloc(cleartext_buffer, 1024); // Resize buffer
3 scrub_memory(cleartext_buffer, 1024); // Attempt to scrub memory
```

`realloc` here is trying to resize a buffer containing sensitive data > The may return a pointer to a different memory location leaving the original memory block intact and accessible to an attacker

2a. How it occurs in a typical program

Confusion around 'freeing' memory. As it does not clear the contents, just marks them available for reuse. Unless explicitly cleared, the data remains in the memory. Certain languages have automation where memory is freed by the runtime, however, once again, the sensitive data must be explicitly cleared. Also reliance on third-party libraries is a common error made.

2b. Impact on the system

If the heap memory is not cleared correctly, the issues that could occur are; data exposure, privilege escalation risks, malware could scan the heap memory searching for patterns looking for sensitive data and insufficient resource usage will impact system performance.

2c. To mitigate

- Always scrub sensitive data before freeing or reallocating memory

- Avoid using `realloc()` for buffers containing sensitive data
- Use secure libraries that scrub data.
- Ensure you use OWASP secure coding practices
- Periodic memory scanning and scrubbing to maintain and prevent vulnerabilities.

```
1 char *cleartext_buffer = get_secret(); // This function retrieves sensitive data
2 scrub_memory(cleartext_buffer, strlen(cleartext_buffer)); // Scrub memory before freeing
3 free(cleartext_buffer); // Free the memory
```

Here the memory is scrubbed before being freed.

[CWE-468: Incorrect pointer scaling](#)

In C and C++ program you could accidentally refer to the wrong memory due to the semantics of when math operations are implicitly scaled.

2a. How it occurs in a typical program

This can happen when a program incorrectly calculates or misuses pointer arithmetic leading to incorrect memory access. Usually common in low level languages where manual memory management and pointer arithmetic are used. Programmers can forget to scale to the correct place and use an incorrect offset causing the pointer to access unintended memory locations, issues with iterating over arrays or buffers can cause the pointer to point beyond the bounds of the array and incorrect data sizes and memory allocations are all further issues that can occur.

2b. Impact on the system.

There are multiple issues that can occur due to the pointer being scaled incorrectly. Memory corruption and unintended data access (incl. crashes), buffer overflows, memory leaks, arbitrary code execution where a scaling error overwrite control data - all of these will cause system instability leading to downtime, unreliable performance and costly trouble shooting.

2c. To mitigate

- Use correct data types and pointer arithmetic, matching pointer types with the data being accessed
- Use array indexing instead of pointer arithmetic
- Implement bounds checking to ensure pointers stay within their limits
- In C++ use safer container classes
- Ensure allocation size accounts for the data type
- Adopt secure coding standards - [CERT C Secure Coding Standard](#) by following specific guidelines
- Code review and pair programming to support each other and catch any pointer errors

Issues and learnings:

With the NASM program - rather than try to code a calculator with a user input, I would of hard coded numbers to be used for the addition. The would have been stored in *section. data*.

Other solutions were tried, but unfortunately many were dis guarded as they were not going to be used (I also encountered numerous issues with these as was programming a calculator) Here is one method I had tried that I happened to have saved:

This was a code I tried on the online assembler, however the result would not display. This seems to be because I failed to convert the registers into strings in order for it to display correctly.

```
1 section .data
2
3     num1 dd 5
4     num2 dd 3
5
6 section .text
7     global _start
8 _start:
9     ; Load values to the registers
10    mov rax, [num1]
11    mov rbx, [num2]
12    ; Add values
13    add rax, rbx
14    ; Exit
15    mov rax, 1
16    int 0x80
17
```

The issue I have with my main code (as seen in Q1) is that the result of the input calculation would not print.

CORRECTION IN CODE

```
home > cfg-vr > Desktop > Assignments > NasmCalc > == calc2.asm
1  default rel
2  section .data
3
4      welcome db 0dh, 0ah, 0dh, 0ah, "***Hey, welcome to this calculator!***", 0dh, 0ah
5      welcome_length equ $ - welcome
6      msg_prompt db "Add your calculation (e.g., 24 + 6): ", 0
7      msg_result db "Result: ", 0
8      msg_newLine db 10, 0
9      invalid_operator db "Invalid input, please use + or - for operations", 0
10
11 section .bss
12     buffer resb 32
13     num1 resq 1 ; Number 1
14     num2 resq 1 ; Number 2
15
16 section .text
17     global _start
18
19 _start:
20     ; Print the welcome message
21     lea rsi, [welcome]
22     mov edx, welcome_length
23     mov eax, 1
24     mov edi, 1
25     syscall
26
27     ; Print the message prompt
28     lea rsi, [msg_prompt]
29     mov edx, 37
30     mov eax, 1
31     mov edi, 1
32     syscall
33
34     ; Read the input
35     lea rsi, [buffer]
36     mov edx, 32
37     xor eax, eax
38     xor edi, edi
39     syscall
40
41     ; Convert first number to an int
42     call parse_number
43     mov [num1], rax
44
45     ; Move to the operator
46     call skip_spaces
47
48     ; Check the operator
49     mov bl, [rsi]
50     inc rsi
51     call skip_spaces
52     call parse_number
53     mov [num2], rax
54     cmp bl, '+'
55     je add_numbers
56     cmp bl, '-'
57     je subtract_numbers
58
59     ; Invalid operator - print error message
60     lea rsi, [invalid_operator]
61     mov edx, 47
```

```
home > cfg-vr > Desktop > Assignments > NasmCalc > == calc2.asm
2  section .data:;_numbers
3
4      ; Invalid operator - print error message
5      lea rsi, [invalid_operator]
6      mov edx, 47
7      mov eax, 1
8      mov edi, 1
9      syscall
10     jmp _exit
11
12     ; Move to the 2nd
13 add_numbers:
14
15     ; Perform addition
16     mov rax, [num1]
17     add rax, [num2]
18     call print_result
19     jmp _exit
20
21     ; Move to the 2nd
22 subtract_numbers:
23     ; Perform subtraction
24     mov rax, [num1]
25     sub rax, [num2]
26     call print_result
27     jmp _exit
28
29 print_result:
30     push rax
31     ; Printing the results
32     lea rsi, [msg_result]
33     mov edx, 8
34     mov eax, 1
35     mov edi, 1
36     syscall
37
38     ; Converting results to a string
39     pop rax
40     jmp print_number
41
42 parse_number:
43     ; Converting a string to a number
44     xor eax, eax
45
46 parse_loop:
47     movzx ecx, byte [rsi]
48     sub cl, '0'
49     cmp cl, '9'
50     ja done_parsing
51     imul rax, rax, 10
52     add rax, rcx
53     inc rsi
54     jmp parse_loop
55 done_parsing:
56     ret
57
58 print_numbers:
59     ; Convert the number in rax to ASCII in buffer and display
60     lea rsi, [buffer + 32]
61     mov ecx, 10
62
63 print_loop:
```

```

home > cfg-vr > Desktop > Assignments > NasmCalc > calc2.asm
111 done_parsing:
112     ret
113
114 print_number:
115     ; Convert the number in rax to ASCII in buffer and display
116     lea rsi, [buffer + 32]
117     mov ecx, 10
118
119 print_loop:
120     xor edx, edx
121     div rcx
122     dec rsi
123     add dl, '0'
124     mov [rsi], dl
125     test rax, rax
126     jnz print_loop
127
128 print_result_buffer:
129     ; display result from buffer
130     lea rdx, [buffer + 32]
131     sub rdx, rsi
132     mov eax, 1
133     mov edi, 1
134     syscall
135     ret
136
137 skip_spaces:
138     ; skipping any spaces in the buffer
139 skip_space_loop:
140     cmp byte [rsi], ' '
141     jne skip_done
142     inc rsi
143     jmp skip_space_loop
144 skip_done:
145     ret
146
147 exit:
148     ; Exit the program
149     mov eax, 60
150     xor edi, edi
151     syscall
152

```

Output:

Addition

▶ Run

📁 Save

Output

Hey, welcome to this calculator!
 Add your calculation (e.g., 24 + 6): Result: 55
 [Execution complete with exit code 0]

Subtraction

▶ Run

📁 Save

Output

Hey, welcome to this calculator!
 Add your calculation (e.g., 24 + 6): Result: 5
 [Execution complete with exit code 0]

Running the code within VSCode still had the issue of not displaying the numerical results, however, using the online compiler, the code works. The amendments were thanks to support from a user on Stack overflow where I asked the question for support, which can be found here: <https://stackoverflow.com/questions/79175900/nasm-calculator-wont-print-the-result/79175956#79175956>

Code available at https://github.com/CharEms-W/Vulnerability_Research/blob/main/calc2.asm

Online compiler <https://www.mycompiler.io/view/DpcUto0LNAl>

References

[Incorrect Pointer Scaling | Martello Security](#)

[NASM Tutorial | Cratecode](#)

[Improper Clearing of Heap Memory Before Release \('Heap Inspection'\) examples in C - Search](#)

[Assembly Language in 100 Seconds](#)

[🌐 Calculator in Assembly Language - Linux x86 & NASM - Division](#)

<https://ubuntuforums.org/showthread.php?t=1930861>

